

Análise do Epidemius: uma calculadora epidêmica baseada no modelo comportamental SEIR

Gabriel Soares de Moraes Vitoriano dos Santos
Aluno do curso de Bacharelado em Sistemas de Informação
Universidade Federal Rural de Pernambuco

10 de maio de 2026

Resumo

Este artigo analisa o funcionamento do Epidemius, uma calculadora epidêmica baseada no modelo comportamental SEIR.

$$\begin{aligned}\frac{dS}{dt} &= -\frac{\beta SI}{N} \\ \frac{dE}{dt} &= \frac{\beta SI}{N} - \sigma E \\ \frac{dI}{dt} &= \sigma E - \gamma I \\ \frac{dR}{dt} &= \gamma I\end{aligned}\tag{1}$$

1 Introdução

No início do ano de 2020, a humanidade foi surpreendida com um evento alarmante denominado pandemia de Covid-19. Essa ocorrência, de proporções mundiais, desencadeou acontecimentos não imaginados pela atual geração, como a suspensão de aulas em de escolas e universidades, fechamento de comércios e interrupção de atividades presenciais nas mais diversas empresas. Simultaneamente, as famílias sentiam a perda e o adoecimento de parentes e de amigos, o que as tornava ansiosas por soluções para o encerramento do cenário pandêmico e a mitigação de sofrimentos.

Dessa forma, a academia teve um papel notório para o enfrentamento da pandemia, e continuará tendo para possíveis novas epidemias. Assim, um dos meios que os pesquisadores dispõem para estudar essas situações de saúde pública é o modelo SEIR, o qual, por meio de equações diferenciais ordinárias, almeja modelar o cenário futuro de uma epidemia, observadas algumas variáveis como tempo de incubação, taxa de internamento e tamanho populacional.

As equações diferenciais do modelo são as apresentadas a seguir:

Em que S indica "indivíduos suscetíveis", E "indivíduos expostos", I "indivíduos infectados" e R "indivíduos recuperados". Ademais, β é a razão entre o número reprodutivo básico ajustado e o tempo em que o paciente está infectado, σ é o inverso do tempo de incubação e γ o inverso do tempo em que o paciente está infectado, e N indica o tamanho da população.

Com isso, criou-se uma calculadora digital baseada nesse modelo.

2 Metodologia

A calculadora Epidemius é inteiramente autoral. Embora tenha havido auxílio de ferramentas de Inteligência Artificial (IA) para sua elaboração, mas unicamente no âmbito de "tutoria", essa aplicação não compromete a origem autoral da calculadora. Além disso, o presente artigo também é autoral, sendo o uso de IA empregado somente para gerar um modelo LaTeX básico e para solucionar dúvidas acerca dessa linguagem de marcação tão presente no meio acadêmico. Nesse sentido, o conteúdo do artigo é original.

Por ser baseada no modelo SEIR, a calculadora necessariamente faz uso das quatro EDOs (equa-

ções diferenciais ordinárias) apresentadas em (1), cuja resolução se dá com o método numérico de Runge-Kutta de quarta ordem.

O desenvolvimento do Epidemius seguiu as seguintes determinações:

- Linguagens: HTML e JavaScript;
- Bibliotecas: Bootstrap para front-end e Chart.JS para gráficos;

3 Funcionamento do código

O código é estruturado em 6 diferentes arquivos, sendo que 5 são JavaScript (.js) e 1 HTML (.html). O arquivo HTML representa a página principal do projeto, onde o usuário, apresentado a um ambiente de formulário, põe os dados das variáveis do modelo sobre os campos de texto (inputs) e clica sobre o botão "Processar dados" para dar início à análise. Dos arquivos JavaScript, o app.js é o principal, possuindo a chamada de um `addEventListener()` sobre o botão de "Processar dados", que serve para dar início ao cálculo com o auxílio da função definida em `seir.js`, de nome `"rungeKuttaSEIR()"`, a qual, por meio do método de Runge-Kutta de quarta ordem, retorna um array com os valores de S, E, I e R no tempo desejado. Tais dados são apresentados na tela ao usar as funções definidas nos arquivos `chart.js` (que não deve ser confundido com a biblioteca de mesmo nome) e `table.js`, que, atuando sobre o HTML da página principal.

Por fim, o último arquivo JavaScript, nomeado `validation.js`, não está diretamente ligado ao modelo, mas é importante para garantir que o usuário não introduza valores estranhos ao processamento.

3.1 Trechos principais

A seguir, está disponível o trecho do código responsável pela execução do método de Runge-Kutta de quarta ordem:

```
1 function rungeKuttaSEIR(t0,
   initialState, tf, derivatives,
   nIter){
2   /* It performs fourth order
      Runge-Kutta method on SEIR.
3     * t0: initial time value
4     * initialState: an array
       containing the initial
```

```
   values of S, E, I and R, in
      this exact order
5   * tf: final time value
6   * derivatives: an array
      containing the derivatives
      dS_dt, dE_dt, dI_dt and
      dR_dt, in this exact order
7   * nIter: number of interactions
8   *
9   * The function returns an
      array containg the values
      of S, E, I and R at the
      final time, in this exact
      order
10  */
11
12  const H = (tf - t0) / nIter;
13
14  let t = t0;
15  let y = [...initialState];
16  let k1, k2, k3, k4;
17
18  for(let i = 0; i < nIter; i++){
19    k1 = derivatives(y[0], y
20      [1], y[2]);
21    k2 = derivatives(y[0] + H *
22      k1[0]/2, y[1] + H * k1
23      [1]/2, y[2] + H * k1
24      [2]/2);
25    k3 = derivatives(y[0] + H *
26      k2[0]/2, y[1] + H * k2
27      [1]/2, y[2] + H * k2
28      [2]/2);
29    k4 = derivatives(y[0] + H *
30      k3[0], y[1] + H * k3
31      [1], y[2] + H * k3[2]);
32
33    for(let j = 0; j < y.length
34      ; j++){
35      y[j] += H * (k1[j] + 2*
36        k2[j] + 2*k3[j] + k4
37        [j]) / 6;
38    }
39
40    t += H;
41  }
42
43  return y;
44 }
```

Listing 1: Função `rungeKuttaSEIR`

Nele, é possível notar o cálculo das quatro variáveis "k" para, por meio da média ponderada delas e do passo, descobrir os próximos valores das funções-

alvo. A função `rungeKuttaSEIR` é chamada dentro de `app.js`, como mostra o seguinte trecho de código:

```
1  const derivatives = (s, e, i) => {
2    const dS_dt = -rt/tinf * i * s
      / n;
3    const dE_dt = rt/tinf * i * s /
      n - e/tinc;
4    const dI_dt = e/tinc - i/tinf;
5    const dR_dt = i/tinf;
6
7    return [dS_dt, dE_dt, dI_dt,
      dR_dt];
8  }
9
10 /* Current state of S, E, I and R:
    */
11
12 let current_state = [n - io, 0, io,
    0];
13
14 /* These are the arrays used to
    store the results: */
15
16 let resultsS = [current_state[0]];
17 let resultsE = [current_state[1]];
18 let resultsI = [current_state[2]];
19 let resultsR = [current_state[3]];
20
21 /* Calculating: */
22
23 const N_ITER = 100;
24
25 for(let t = 0; t < ts; t++){
26   current_state = rungeKuttaSEIR(
      t, current_state, t+1,
      derivatives, N_ITER);
27
28   resultsS.push(current_state[0])
      ;
29   resultsE.push(current_state[1])
      ;
30   resultsI.push(current_state[2])
      ;
31   resultsR.push(current_state[3])
      ;
32 }
```

Listing 2: Trecho de `app.js`

várias famílias perdendo entes e amigos queridos. Ela fez a sociedade tomar mais consciência de epidemias, e, havendo mais consciência, os estudos na área têm se tornado mais fecundos. A implementação do modelo SEIR feita com o `Epidemius` sinaliza um apoio aos estudos na área.

4 Conclusão

A pandemia de Covid-19 foi, certamente, um dos eventos mais traumáticos dos últimos tempos, com